

ArgoCD App of Apps

What is the “App of Apps” Pattern?

The “App of Apps” pattern in ArgoCD allows you to define a parent application that references child applications. This hierarchical structure simplifies the management of complex environments by enabling you to control multiple applications through a single entry point. Each child application can be managed independently while being part of the larger deployment strategy.

Benefits of the “App of Apps” Pattern

- **Centralized Management:** Manage all your applications from a single point.
 - **Single Pane of Glass:** Administrators can view and control all applications through the ArgoCD dashboard, reducing the need to switch contexts.
 - **Simplified Access Control:** Access permissions and policies can be centrally managed, ensuring consistent security practices across all applications.
 - **Unified Monitoring:** Centralized monitoring and logging allow for quicker identification and resolution of issues, as all application states are visible from one place.
- **Consistency:** Ensure consistent application deployment and configuration.
 - **Automated Sync Policies:** ArgoCD’s automated sync policies help keep the deployed state in line with the desired state, automatically correcting any drift.
- **Scalability:** Easily scale the number of applications managed by ArgoCD.
 - **Horizontal Scaling:** New applications can be added to the manifests directory and referenced in the parent application without disrupting the existing setup, allowing seamless horizontal scaling.
 - **Cluster Expansion:** Adding new clusters or environments becomes straightforward as the same parent application can manage deployments across multiple clusters.
- **Modularity:** Maintain modular and reusable application definitions.
 - **Reusability:** Each child application is defined in its own YAML file, making it easy to reuse and share across different projects or teams.
 - **Ease of Maintenance:** Updates and modifications can be made to individual child applications without affecting the overall system, reducing the risk of downtime.
 - **Disaster Recovery and Migration:** In case of cluster failures or migrations, the modular structure allows for quick replication and redeployment of applications. You can easily

redeploy the entire set of applications to a new cluster by applying the parent application manifest.

Prerequisites

Before we dive into the implementation, ensure you have the following:

- A running Kubernetes cluster.
- ArgoCD installed in your cluster.
- A GitHub repository to store your application definitions.

Guide

Setting Up the GitHub Repository

Create a GitHub repository to store your ArgoCD application manifests. For this example, we will use the following structure:

Step1

create projects of each stack

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/infra-appstack-project.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/security-appstack-project.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/servicemesh-appstack-project.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/observability-appstack-project.yaml>

ex: <https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/infra-appstack-project.yaml>

Example file: create separate file for each project under <[gke-bootstrap](#) repo >

infra-appstack

observability-appstack

servicemesh-appstack

security-appstack

apiVersion: argoproj.io/v1alpha1

kind: AppProject

metadata:

name: infra-appstack

namespace: argocd

finalizers:

- [resources-finalizer.argocd.argoproj.io](#)

spec:

description: infra-appstack

sourceRepos:

- ''

destinations:

- namespace: ''

server: <https://kubernetes.default.svc>

clusterResourceWhitelist:

- group: ''

kind: ''

namespaceResourceWhitelist:

- group: ''

kind: ''

Step2

Create Appstack for each stack and assign related projects

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/observability-appstack.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/servicemesh-appstack.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/security-appstack.yaml>

<https://github.com/cvs-health-source-code/gke-bootstrap/blob/appofapps/test/manifests/infra-appstack.yaml>

Defining the Parent Application

Create a `applications-stack.yaml` file that defines the parent application. This file will reference the child applications.

Created a Parent App for each category

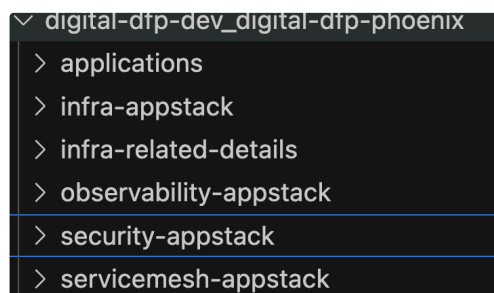
structure:

ex: `digital-dfp-dev_digital-dfp-phoenix/observability-appstack`

`digital-dfp-dev_digital-dfp-phoenix/security-appstack`

`digital-dfp-dev_digital-dfp-phoenix/servicemesh-appstack`

`digital-dfp-dev_digital-dfp-phoenix/infra-appstack`



Create `app1.yaml` and `app2.yaml` files in the `manifests` directory for the child applications. Each file will contain the specific configuration for an application.

```

  security-appstack
    ! cert-manager.yaml
    ! constraint-templates.yaml
    ! external-secrets.yaml
    ! gatekeeper-policies.yaml
    ! opa-gatekeeper.yaml
    ! platform-services-clustersecretstore...
    ! twistlock.yaml
  servicemesh-appstack
    ! istio-cr.yaml
    ! istio-external-gateway.yaml
    ! istio-internal-gateway.yaml
    ! istio-ns.yaml
    ! istio-operator.yaml
    ! istio-telemetry.yaml
    ! kiali-operator.yaml

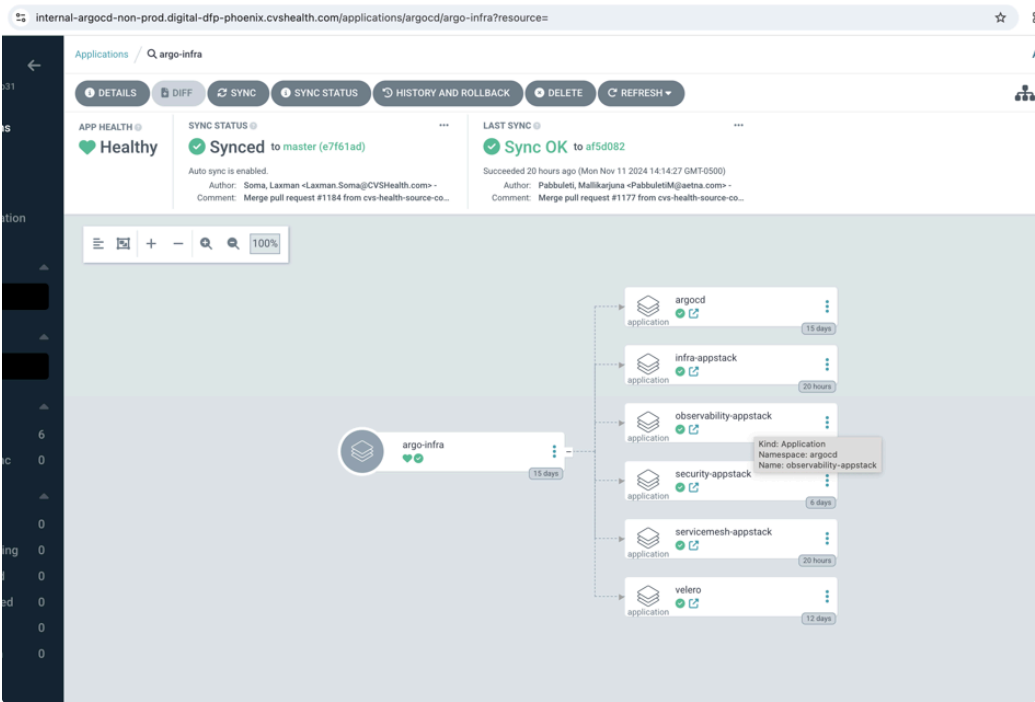
```

```

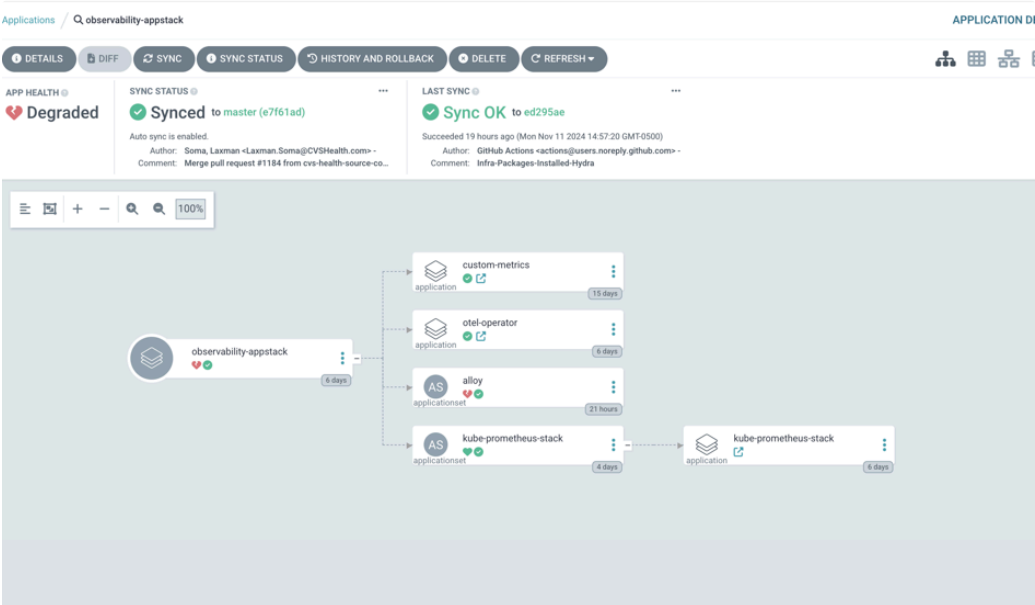
  infra-appstack
    ! config-connector.yaml
    ! dns-masq-agent.yaml
    ! harness-delegate-ng-latest.yaml
    ! k8s-api-proxy.yaml
    ! post-rollout-cluster-ops.yaml
    ! priority-class-global.yaml
    ! reflector.yaml
    ! reloader.yaml
    > infra-related-details
  observability-appstack
    ! alloy.yaml_bk
    ! alloygitgenerator.yaml
    ! custom-metrics.yaml
    ! kube-prometheus-stack.yaml
    ≡ kube-prometheus-stack.yaml_bk
    ! otel-operator.yaml

```

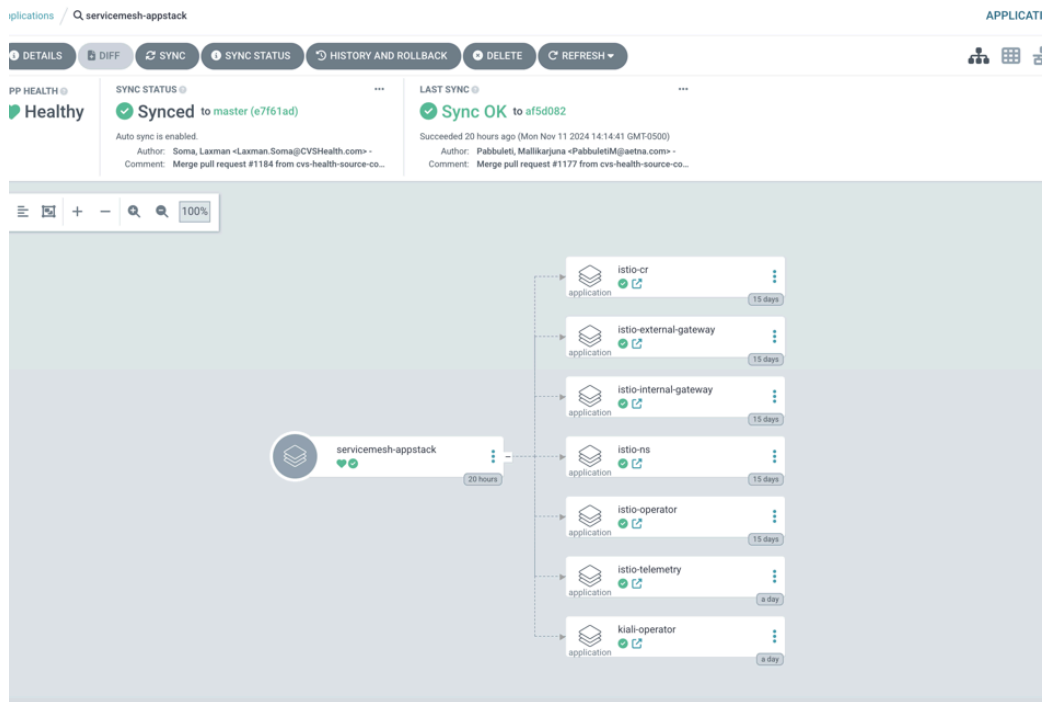
Verifying in Arogoacd agocd-infra app



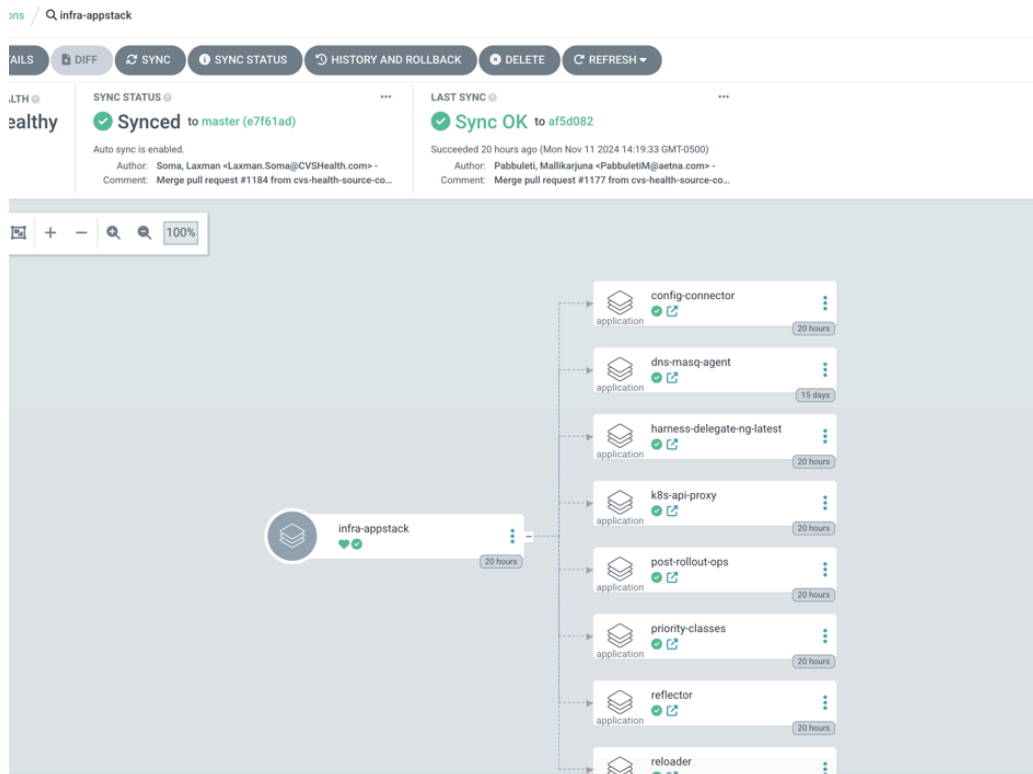
Observability App Stack



ServiceMesh App Stack:



InfraAppStack



update hydra scripts and generated manifest files to copy files to related directory structure.

Note:

For Existing Apps to move in Parents apps , Application will prune and recreate in new directory , it is recommended to use for new cluster builds .